# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA17**

---

**COURSE OUTCOME COVERED**

**CO5:** *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

*Assessment Tool Weightage: 50%*

---

**Duration : 1 Hour**

**QUESTIONS: 5** **Total Marks:50**

Annexure A

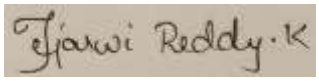
## Design Task

---

***Code of Conduct:***

*I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***



Annexure A

# Design Task

1. **Hybrid AI Recommendation Engine Design**
   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

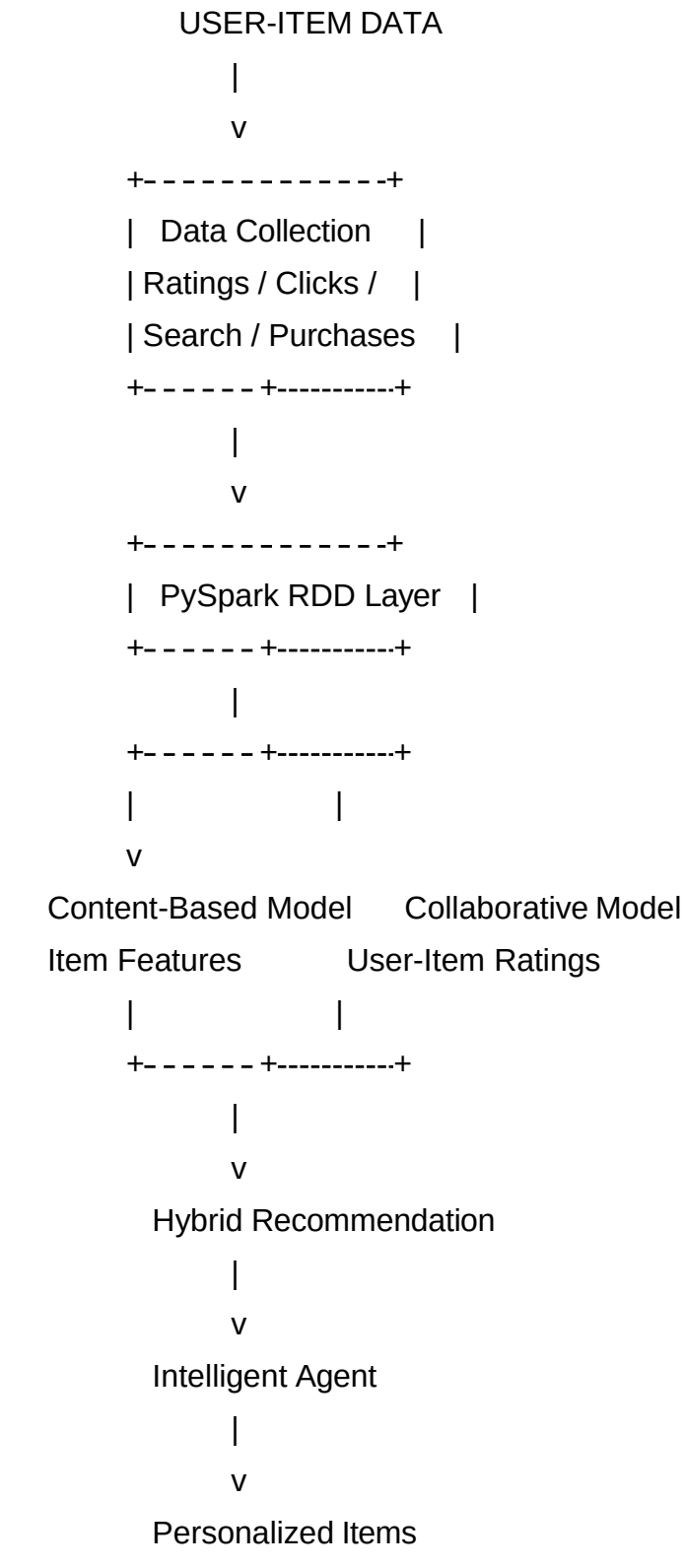
Design Tasks – Distributed AI Systems Using PySpark RDDs and Intelligent Agents

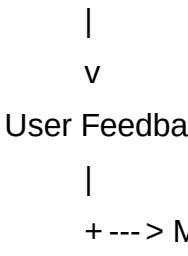
1. Hybrid AI Recommendation Engine Design

Objective

Design a scalable recommendation system that combines Content-Based Filtering and Collaborative Filtering using PySpark RDDs.

System Architecture






Content-Based Filtering

The system represents each item using features such as:

- Category
- Keywords
- Price
- Brand
- Description

RDD operations can transform item information:

```
items = sc.textFile("items.csv")

item_features = items.map(
    lambda x: x.split(",")
)

category_items = item_features.map(
    lambda x: (x[1], x[0])
)
```

The system recommends items similar to those the user previously liked.

Collaborative Filtering

Collaborative filtering uses the behavior of similar users.

```
ratings = sc.textFile("ratings.csv")

user_item = ratings.map(
    lambda x: x.split(",")
).map(
    lambda x: (x[0], x[1], float(x[2]))
)
```

RDD operations such as map(), filter(), groupByKey(), reduceByKey() and join() can process large user-item datasets in parallel.

Hybrid Recommendation

The final score can be calculated as:

[
Score=\alpha C+(1-\alpha)CF
]

where:

- (C) = content-based score
- (CF) = collaborative-filtering score
- (\alpha) = weighting factor

For example:

[
Score=0.4C+0.6CF
]

Role of Intelligent Agents

An intelligent recommendation agent:

1. Observes clicks, searches, purchases and ratings.
2. Updates the user's preference profile.
3. Detects changes in user interests.
4. Generates new recommendations.
5. Learns from user feedback.

Advantages

- Scalable to millions of users.
- Handles large interaction datasets.
- Combines two recommendation approaches.
- Provides personalized recommendations.
- Agents enable real-time adaptation.

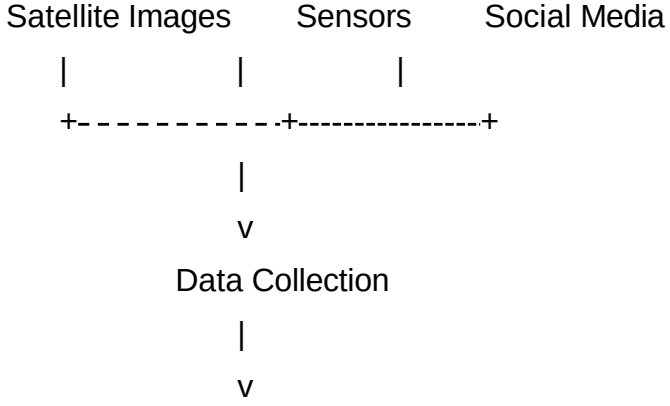
---

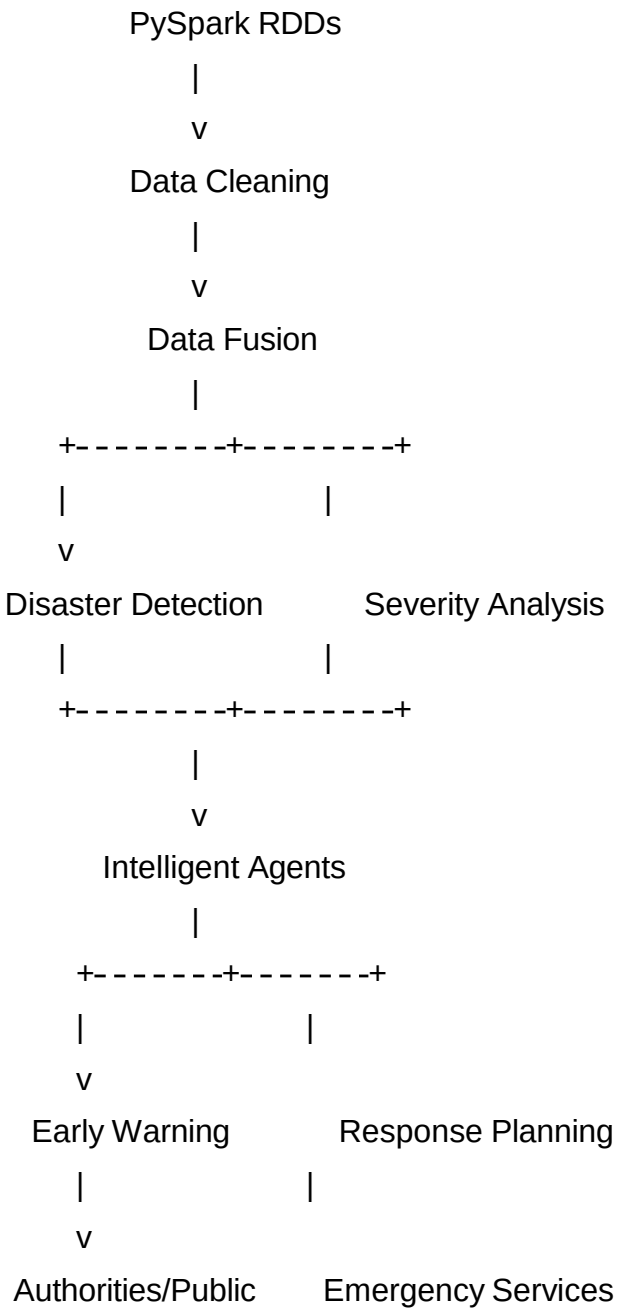
2. Smart Disaster Detection System

Objective

Develop a distributed AI system for detecting floods, earthquakes and other disasters using satellite data, sensor data and social-media information.

System Architecture






Data Sources

| Source | Information |
|---|---|
| Satellite | Flood extent, temperature, land changes |
| Sensors | Water level, seismic activity, rainfall |
| Social media | Disaster reports and location information |
| Weather data | Rainfall, wind and temperature |

PySpark RDD Processing

```
sensor_data = sc.textFile("sensor_data.csv")
```

```
clean_data = sensor_data.filter(
    lambda x: x is not None
)

sensor_values = clean_data.map(
    lambda x: x.split(",")
)

water_levels = sensor_values.map(
    lambda x: (x[0], float(x[2]))
)

high_levels = water_levels.filter(
    lambda x: x[1] > 8.0
)
```

Data Fusion

Different sources are combined to improve reliability.

For example:

$$
\begin{aligned}
&[\\
&\text{DisasterScore} = \\
&w_1(\text{Sensor})+w_2(\text{Satellite})+w_3(\text{SocialMedia})\\
&]
\end{aligned}
$$

If several independent sources report the same event, confidence increases.

Intelligent Agent Coordination

Different agents can perform different tasks:

- Sensor Agent: monitors sensor readings.
- Satellite Agent: analyzes satellite observations.
- Social Media Agent: detects disaster-related messages.
- Fusion Agent: combines evidence.
- Warning Agent: generates alerts.
- Response Agent: recommends evacuation or emergency actions.

Early Warning

Sensor anomaly

↓

Satellite confirmation

↓

Social-media confirmation

↓

Fusion Agent

↓

High confidence

↓

Warning Agent

↓

Emergency Alert

Advantages

- Rapid disaster detection.
- Distributed processing of huge datasets.
- Multiple information sources increase reliability.
- Agents can coordinate automatically.
- Supports early evacuation and emergency response.

---

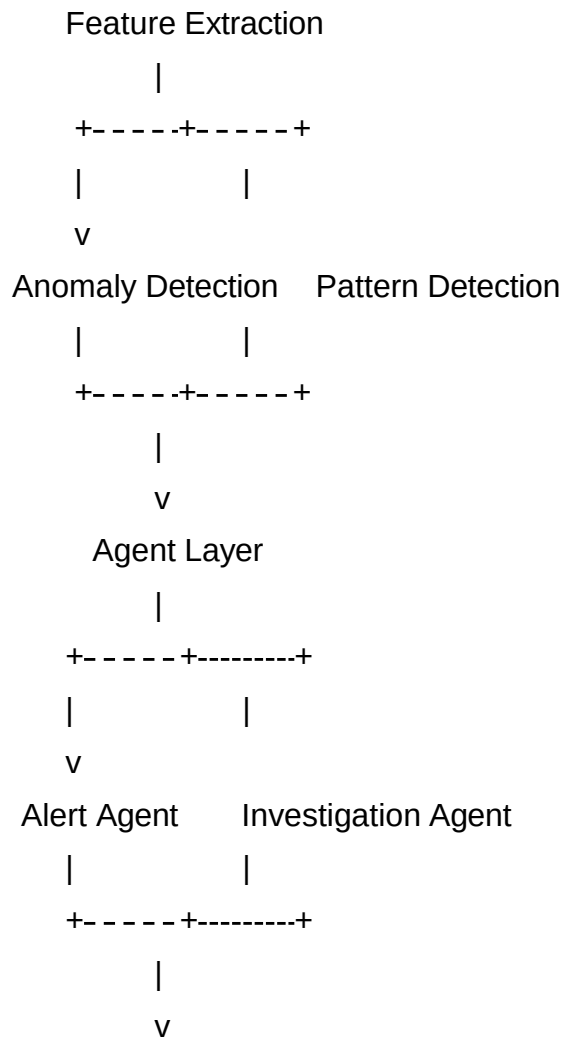
3. AI-Based Fraud Detection Framework

Objective

Design a scalable system that detects fraudulent financial transactions using PySpark RDDs and intelligent agents.

Architecture

```
Financial Transactions
        |
        v
+----------+
| Data Ingestion  |
+-----+-----+
        |
        v
+----------+
| PySpark RDDs    |
+-----+-----+
        |
        v
Data Preprocessing
        |
        v
```




Transaction Features

- Transaction amount
- Time
- Location
- Device
- Merchant
- Frequency
- Previous transaction behavior

RDD Processing

```
transactions = sc.textFile("transactions.csv")

data = transactions.map(
    lambda x: x.split(",")
)
```

```
large_transactions = data.filter(
    lambda x: float(x[2]) > 100000
)


merchant_total = data.map(
    lambda x: (x[3], float(x[2]))
).reduceByKey(
    lambda a, b: a + b
)
```

Anomaly Detection

A transaction can be given an anomaly score:

$$
A=w_1X_1+w_2X_2+w_3X_3+\cdots+w_nX_n
$$

If:

$$
A > Threshold
$$

the transaction is considered suspicious.

Intelligent Agents

Transaction Agent

Monitors incoming transactions.

Anomaly Agent

Identifies unusual behavior.

Pattern Agent

Detects repeated fraud patterns.

Alert Agent

Generates alerts.

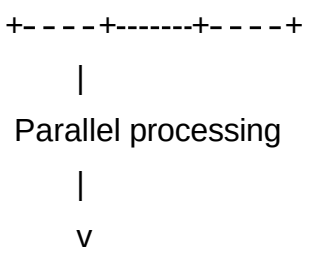
Learning Agent

Uses confirmed fraud cases to improve future detection.

Distributed Processing

RDDs divide transactions into partitions so that multiple machines can process them simultaneously.

```
10 million transactions
        |
        v
+- - - -+-------+- - - -+
| Partition 1   | P2    | P3 ...
```




Advantages

- Handles millions of transactions.
- Supports parallel processing.
- Detects suspicious behavior quickly.
- Agents continuously improve detection.
- Reduces financial losses.

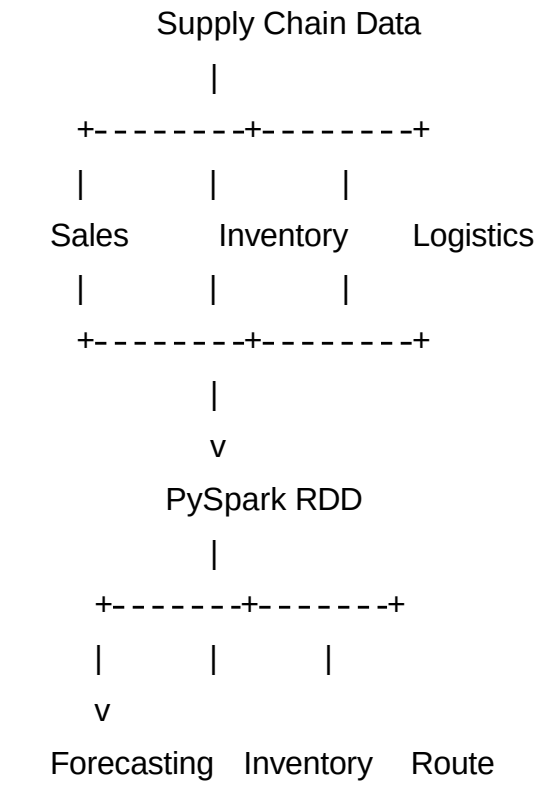
---

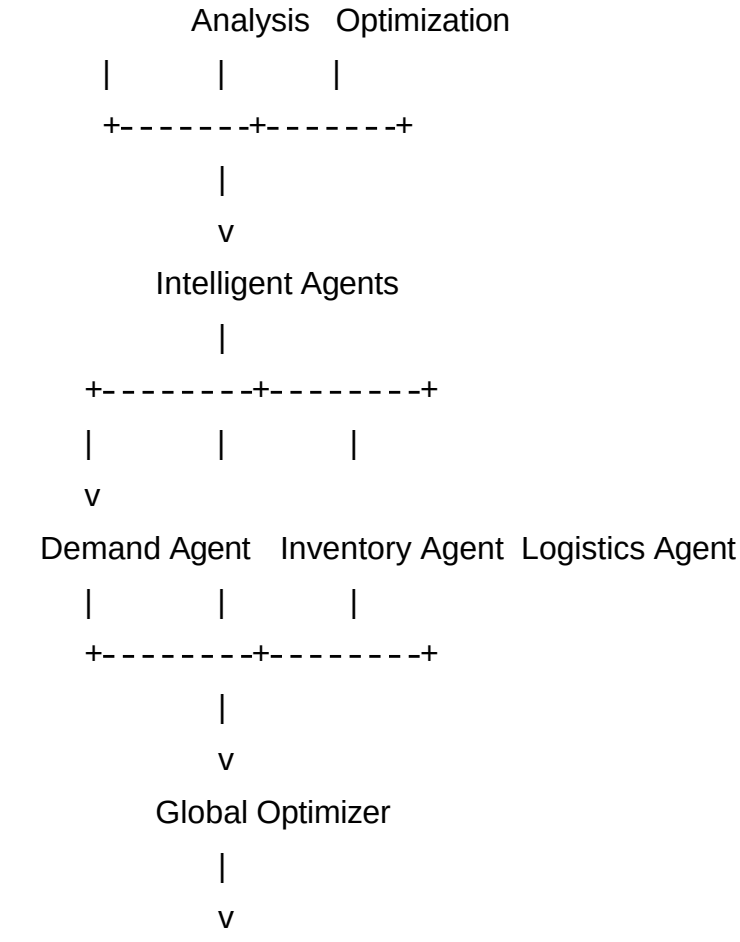
4. Autonomous Supply Chain Optimization System

Objective

Design a distributed AI system for inventory management, logistics optimization and demand forecasting.

System Architecture






RDD Transformations

RDD transformations include:

- map()
- filter()
- flatMap()
- reduceByKey()
- groupByKey()
- join()

Example:

```
sales = sc.textFile("sales.csv")

product_sales = sales.map(
    lambda x: x.split(",")
).map(
    lambda x: (x[1], int(x[2]))
)
```

```
total_sales = product_sales.reduceByKey(
    lambda a, b: a + b
)
```

The resulting sales information can be used for demand forecasting.

Intelligent Agents

Demand Agent

Predicts future product demand.

Inventory Agent

Maintains appropriate stock levels.

Logistics Agent

Selects efficient delivery routes.

Supplier Agent

Monitors supplier availability and delivery time.

Coordination Agent

Coordinates decisions between the other agents.

Decentralized Decision-Making

For example:

Demand Agent → High future demand

↓

Inventory Agent → Increase stock

↓

Supplier Agent → Order additional units

↓

Logistics Agent → Select fastest low-cost route

Objective Function

The system attempts to minimize:

$$
\begin{aligned}
&\text{Total Cost} = \\
&\text{Inventory Cost} + \\
&\text{Transportation Cost} + \\
&\text{Ordering Cost} + \\
&\text{Shortage Cost}
\end{aligned}
$$

while maintaining customer service levels.

Advantages

- Reduces inventory costs.
- Improves demand forecasting.
- Optimizes delivery routes.

- Supports decentralized decisions.
- Handles large-scale supply-chain data.

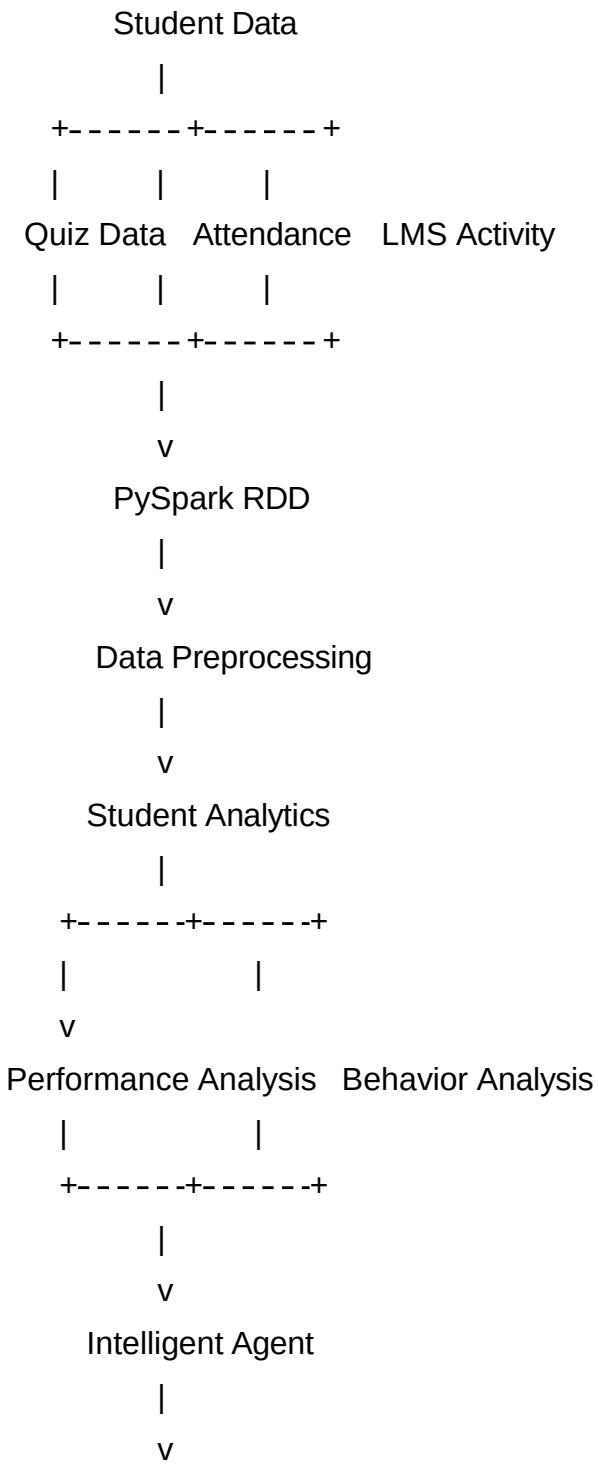
---

5. Personalized Learning Analytics Platform

Objective

Develop an AI-driven educational platform that analyzes student behavior and provides personalized learning using PySpark RDDs and intelligent agents.

System Architecture



Personalized Content

|

v

Student

|

v

Feedback

|

+----- > Agent Learning

Data Collected

- Quiz scores
- Assignment marks
- Time spent on lessons
- Video completion
- Number of attempts
- Attendance
- Learning pace
- Topic-wise performance

RDD Processing

```
students = sc.textFile("student_data.csv")

student_data = students.map(
    lambda x: x.split(",")
)

scores = student_data.map(
    lambda x: (x[0], float(x[2]))
)

average_scores = scores.groupByKey().mapValues(
    lambda values: sum(values) / len(values)
)
```

Student Classification

Students can be classified as:

High Performer

|

+---- Advanced Content

Average Performer

|

+---- Regular Content

Low Performer

|

+---- Remedial Content

Intelligent Agents

Student Agent

Maintains the learner's profile.

Content Agent

Selects appropriate learning materials.

Assessment Agent

Analyzes quiz and assignment performance.

Feedback Agent

Provides immediate feedback.

Recommendation Agent

Recommends the next topic or activity.

Adaptive Learning

Suppose a student repeatedly performs poorly in Python loops.

Poor quiz performance

↓

Student Agent detects weakness

↓

Content Agent recommends basic loop lesson

↓

Student completes practice

↓

Assessment Agent evaluates result

↓

Feedback Agent provides feedback

↓

Recommendation updated

Personalized Recommendation Score

A simple score can be calculated as:

$$
R=w_1Performance+w_2Interest+w_3Difficulty+w_4Progress
$$

The highest-scoring content is recommended.

Scalability

PySpark allows thousands or millions of student interactions to be distributed across multiple worker nodes.

Student Records

|

v

RDD Partitions

+---+---+---+

| P1 | P2 | P3 |

+---+---+---+

| | |

Worker

| | |

+---+-----+

|

v

Personalized Results

Advantages

- Personalized learning paths.
- Real-time feedback.
- Scalable to thousands of learners.
- Identifies weak topics early.
- Reduces manual analysis for teachers.
- Continuously adapts to student behavior.

---

Overall Comparison

| System | Main AI Technique | PySpark Role | Intelligent Agent Role | Main Benefit |
|---|---|---|---|---|
| Recommendation Engine | Content + Collaborative Filtering | Process user-item interactions | Adapt recommendations | Personalization |
| Disaster Detection | Anomaly Detection + Data Fusion | Process multi-source data | Coordinate warnings | Early disaster response |

| System | Main AI Technique | PySpark Role | Intelligent Agent Role | Main Benefit |
|---|---|---|---|---|
| Fraud Detection | Anomaly Detection | Process transactions | Detect and investigate fraud | Financial security |
| Supply Chain | Forecasting + Optimization | Process sales/logistics data | Decentralized decisions | Cost reduction |
| Learning Analytics | Predictive Analytics + Recommendation | Process student activity | Adapt learning content | Personalized education |

Common Distributed AI Architecture

All five systems follow the same basic principle:

Large-Scale Data

↓

PySpark RDD

↓

Distributed Processing

↓

AI/ML Analysis

↓

Intelligent Agents

↓

Real-Time Decision

↓

User/System Feedback

↓

Continuous Learning

Conclusion: PySpark RDDs provide the distributed data-processing layer required to handle large-scale datasets, while intelligent agents provide the decision-making and adaptive layer. Combining both creates scalable AI systems capable of continuously observing data, analyzing it, taking actions, and adapting to new feedback.